

Document outline

Simulation framework	1
Dependencies	1
Structure	2
Datasets	2
Execution	2
Implementation choices	2
Bibliography	5

Simulation framework

The Python package performs simulations on large networks and large datafiles. The simulations are therefore highly optimized for scalability. This is done by effectively combining the procedural execution of Python code with columnar and graph based data structures. This format allows large scale data-analysis to take advantage of optimized database queries, network traversal algorithms to take advantage of the graph based data structure, and sampling algorithms to take advantage of procedural executions. With the combination of procedural, columnar and graph based software, every algorithm used by the simulation can be executed with the ideal data-structure. This greatly increases the simulation's scalability.

Dependencies

The columnar and graph based data structures used to increase simulation scalability are provided by the DuckDB, and OSMnx.

DuckDB is an source embedded database designed for high performance analytical (OLAP) workloads (Raasveldt & Mühleisen, 2019). Integration of DuckDB into the Python package is done using the DuckDB Python API, increasing package maintainability. An in-memory database is used. The networks, POI and bus-stops are extracted from OpenStreetMap using OSMnx, a open-source Python package capable of downloading, modeling, analyzing and visualizing any data from OpenStreetMap (Boeing, 2025). Networks downloaded through OSMnx are stored using the NetworkX data structure for graphs. This data structure is highly optimized for graph traversal algorithms, and allows high speed communication with the database.

The package makes use of certain Python libraries in addition to DuckDB and OSMnx. These libraries are mainly used to communicate between the different data representations. A up-to-date list of these dependencies can be found in PyPI.

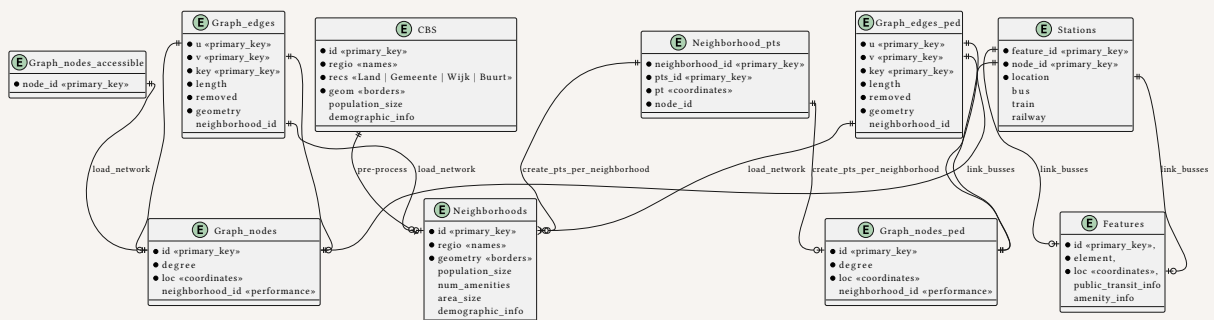


Figure 2: The database used by the package. Not all tables exist on initialization of the Database class. Temporary tables with intermediate results are not shown.

Structure

The package is designed to be maintainable and interpretable. For this reason a standard package structure was applied. All simulations are performed using the main simulation class. This class contains methods to select a city and perform the pre-defined simulations.

The main class makes use of the settings class (see Figure 4). This data-class contains all configurable parameters used during the simulation. Separate fields of this data-class consist of a default value, a current value and a description. The settings can be configured by simply assigning a new value to a field. The class can be converted to string and Pandas DataFrame for easy inspection. Furthermore it also contains a method to restore the default configuration. Default functions used by the settings class are also provided to allow the functions parameters to be altered.

The Database and Network classes act as wrappers for the database and networks respectively. Any communication with the database or networks is done through the methods of this class. These classes are used in the backend of the simulation and are thus not needed by the user.

Datasets

Due to the size of the datasets exceeding API limits some datasets should be downloaded manually. Namely: The “Kerncijfers wijken en buurten 2024” (translated: Core-numbers districts and neighborhoods 2025) (Centraal Bureau voor de Statistiek, 2025a) and the “Wijk- en buurtkaart 2025” (translated: districts and neighborhoods map 2025) (Centraal Bureau voor de Statistiek, 2025b). These datasets contain information about the neighborhoods, including the sizes of the different demographic groups, and the neighborhood borders.

The pedestrian network, car-accessible network, POI and bus-stops are all downloaded using the OSMnx wrapper for the OpenStreetMap API. Downloading using API can take a long time. The simulation therefore includes an option to write this information to files, saving time in following simulations.

Execution

When simulating using this package, the first step is always to initialize the Simulator class with the paths to the CBS datasets. Internally this creates the CBS table with the data from the datasets (see Figure 2).

After initialization the user only needs to call the correct simulation method with the correct parameters. Internally this generally performs the following actions:

1. Load the cities pedestrian network, car-accessible network, POI and stations into the database.
2. Pre-process the data creating tables storing any information that is needed multiple times later.
3. Create sample points within the bounds of the city (see Figure 7). These points represent the source locations when walking to transit stops, and are used during the simulation(s) to calculate the average distance to the nearest transit stop.
4. Pedestrianize the network
5. Move transit stops accordingly
6. Visualize the results. The generated images are stored in a file of choice.
7. Return the results for optional post-processing.

Pedestrianization and the according movement of transit stops may be repeated a number of times to simulate the effects of different levels of pedestrianization.

Implementation choices

When simulating the distance to the nearest transit stop, some implementation choices have an impact on the results. These settings are configurable in the Python package. This section explains the substantiates the choices made for this studies simulation.

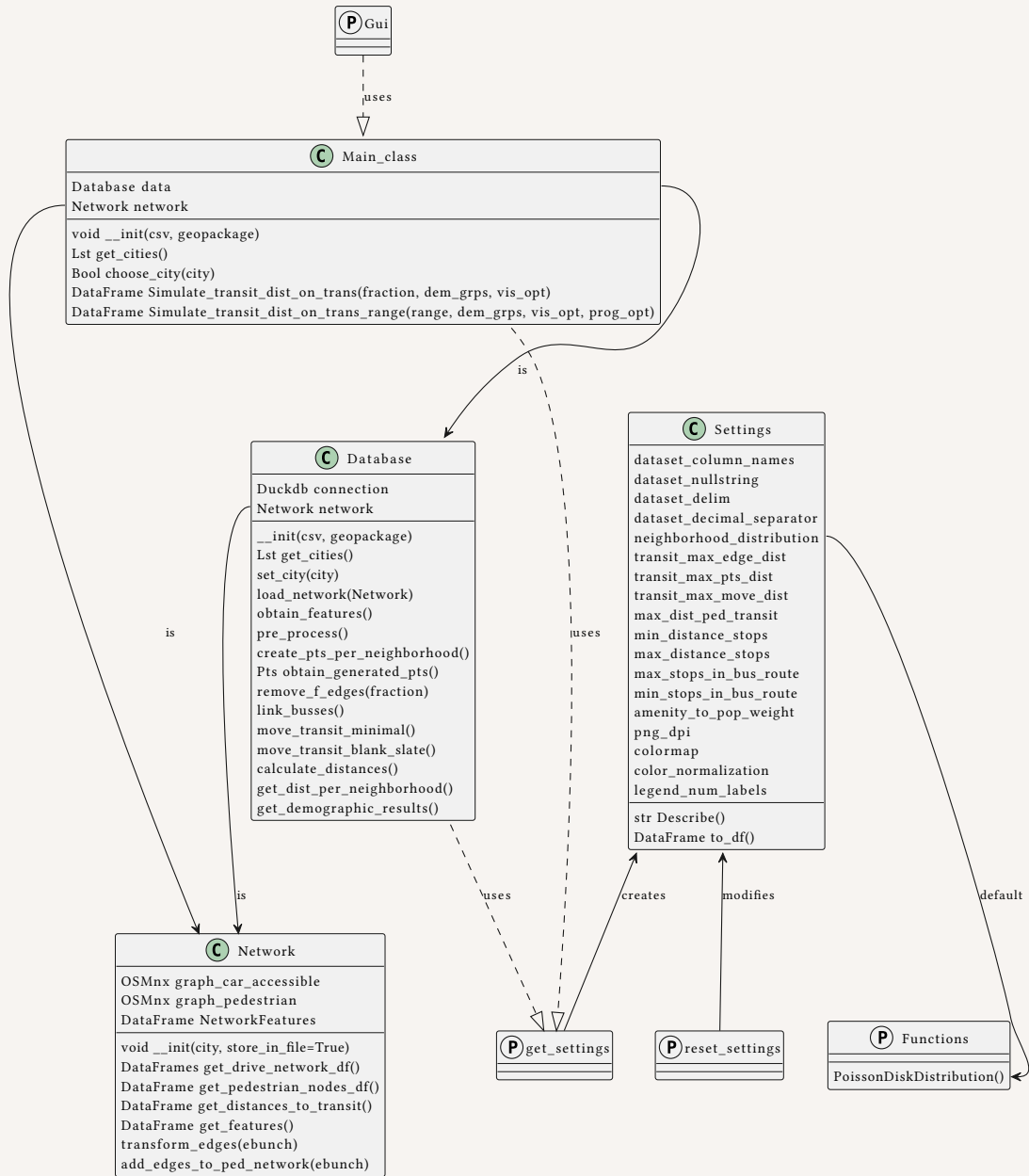


Figure 4: The UML describing the basic structure of the package.

Generate sample points:

To determine the average distance the average person has to travel to reach the nearest transit stop some source points had to be defined. The average distance of the neighborhood to the nearest transit stop would then be the average distance from the source points to the nearest transit stop. In accordance to the research presented in this study, the sample points are to represent the population living in the neighborhood.

The point generation method:

In this simulation the sample points are generated using the Poisson disk distribution with a radius of thirty meters and seven candidates per iteration (Bridson, 2007). This results in a blue-noise distribution (see Figure 7). The randomness in this distribution prevents the directional bias present in grid based distributions while the minimal spacing prevents the clumping found in fully random distributions. Furthermore, the large neighborhoods are represented with more points to ensure accuracy between neighborhoods is preserved. The Poisson disk distribution is therefore able to

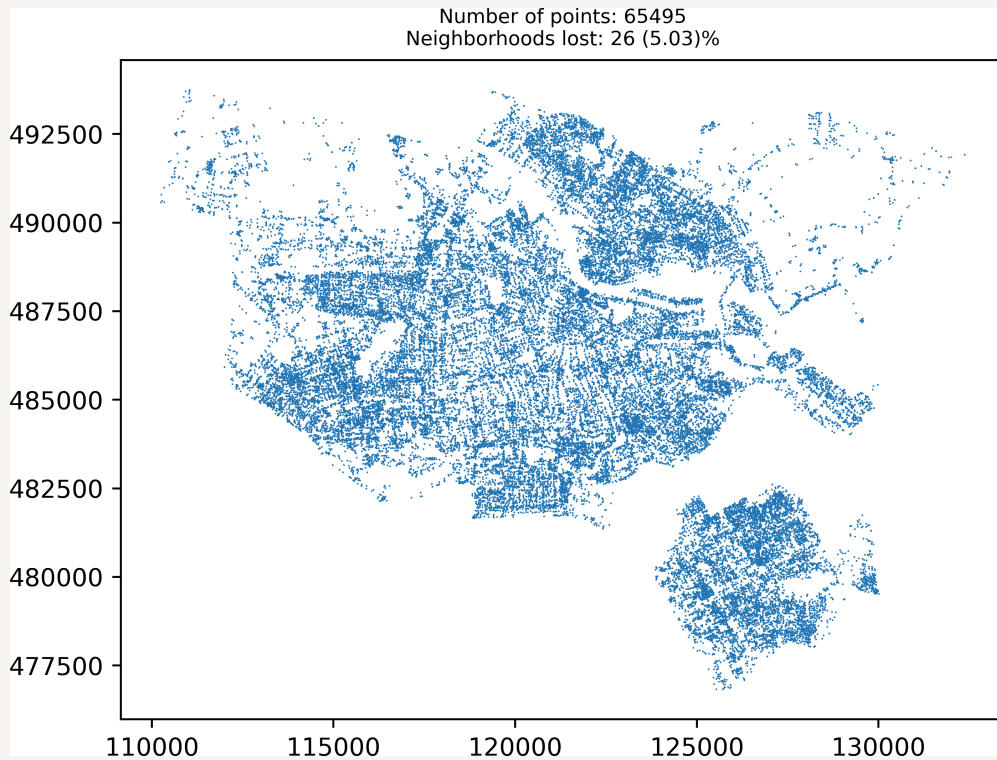


Figure 7: Points generated in Amsterdam using Poisson disk sampling with a radius of thirty meters and seven candidates per iteration. Points outside city borders or further then thirty meters from the pedestrian network are considered to be in a unlivable area and thus rejected.

accurately represent the neighborhoods. In practice a smaller radius results in more points, and thus more accurate results. A radius of thirty was chosen after carefully considering trade-offs between performance and result accuracy.

Point rejection:

The points generated by the Poisson disk sampling are distributed over the entire bounding box of a neighborhood. Points not within the bounds of the neighborhood, or not within a livable area are not a accurate representation of the neighborhood population are therefore rejected. Because a livable area is usually characterized by pedestrian streets, any sample point located further away than thirty meters from the pedestrian network are rejected. Due to the radius of the Poisson disk sampling, a lower margin might lead to strokes of pedestrian network without points assigned. As shown in Figure 7 this accurately rejects any points in unlivable area's like the canals and the Rhine River.

OpenStreetMap deviation buffer:

Bus-stops are extracted from OpenStreetMap. However in OpenStreetMap bus-stops are not necessarily integrated in the car-accessible and pedestrian network. This integration is thus performed by the simulation. As bus-stops are often located a small distance from the road and pedestrian network a small buffer should be used when integrating bus-stops to the pedestrian and car-accessible networks. This study uses a buffer of thirty meters for this. Any bus stop not within a radius of thirty meters to both the pedestrian and car-accessible networks are rejected.

Bus-stops per route:

The algorithm moving bus-stops by completely redesigning the bus network creates new bus-routes by iteratively selecting new bus-stops, stopping only when the destination has been reached, or no new valid bus-stop is available. However depending on the distance between the sources and destinations this might lead to hundreds of bus-stops in a single route. Such routes obviously

wouldn't be used in practice. Therefore a minimum and maximum number of stops in a single route have been introduced. Routes are stopped when this maximum has been reached. Routes with less bus-stops than this minimum are rejected. The number of stops in a bus-route is not fixed and differs between cities and routes. Some margin is therefore used when selecting the minimum and maximum number of stops. This study therefore uses a minimum of 9 stops and a maximum of 30 stops.

Bibliography

- Boeing, G. (2025). Modeling and analyzing urban networks and amenities with OSMnx. *Geographical Analysis*, 57(4), 567–577. <https://doi.org/10.1111/gean.70009>
- Bridson, R. (2007). Fast Poisson disk sampling in arbitrary dimensions. *SIGGRAPH Sketches*, 10(1), 1.
- Centraal Bureau voor de Statistiek. (2025a,). *Kerncijfers wijken en buurten 2025*. CBS. <https://www.cbs.nl/nl-nl/maatwerk/2025/40/kerncijfers-wijken-en-buurten-2025>
- Centraal Bureau voor de Statistiek. (2025b,). *Wijk- en buurtkaart 2025 versie 1*. CBS. <https://www.cbs.nl/nl-nl/dossier/nederland-regionaal/geografische-data/wijk-en-buurtkaart-2025>
- Raasveldt, M., & Mühleisen, H. (2019). DuckDB: an Embeddable Analytical Database. *Proceedings of the 2019 International Conference on Management of Data*, 1981–1984. <https://doi.org/10.1145/3299869.3320212>